

Algoritmi e Strutture Dati
Elaborato a.a. 2025–26

Problema dello Zaino 0/1

Risoluzione con programmazione dinamica
e confronto con un solver ILP

Marco Cesari

2 settembre 2026

Indice

1	Introduzione e definizione del problema	2
1.1	Il problema	2
1.2	Formulazione come problema di ottimizzazione	2
1.3	Perché lo Zaino 0/1	2
2	Applicabilità della programmazione dinamica	3
2.1	I sottoproblemi	3
2.2	Caratterizzazione della struttura di una soluzione ottima	3
2.3	Definizione ricorsiva del valore di una soluzione ottima	4
2.4	Risoluzione ripetuta dello stesso sottoproblema	4
3	I due algoritmi	4
3.1	Calcolo del valore di una soluzione ottima	5
3.2	Costruzione di una soluzione ottima	5
4	Analisi asintotica	5
4.1	Tempo di esecuzione	6
4.2	Occupazione di memoria	6
4.3	Natura pseudo-polinomiale	6
5	Implementazione e varianti	6
5.1	Struttura del codice	6
5.2	La variante a spazio ridotto	7
5.3	Scelte tecnologiche e implementative	7
6	Sperimentazione	8
6.1	Istanze, metodo e piattaforma	8
6.2	Crescita empirica e analisi asintotica	8
6.3	Limiti pratici	11
6.4	Confronto con il solver	12
6.5	Riproduzione degli esperimenti	12
7	Limitazioni e conclusioni	12
7.1	Limitazioni riscontrate	12
7.2	Conclusioni	13

1 Introduzione e definizione del problema

Questo elaborato affronta il problema dello Zaino 0/1 (*0/1 Knapsack*), un classico problema di ottimizzazione combinatoria, risolvendolo con il paradigma della programmazione dinamica.

1.1 Il problema

Sono dati n oggetti; ciascun oggetto i (con $i = 1, \dots, n$) ha un peso $w_i > 0$ e un valore $v_i > 0$, entrambi interi. È inoltre data la capacità W dello zaino, un intero positivo. Si vuole selezionare un sottoinsieme di oggetti che massimizzi il valore totale rispettando il vincolo che il peso totale non superi W . Ogni oggetto può essere incluso una volta sola oppure escluso del tutto (interamente o per niente): da qui il nome *0/1*, che distingue il problema dalla variante frazionaria in cui è ammesso prendere porzioni di un oggetto.

1.2 Formulazione come problema di ottimizzazione

Introducendo per ogni oggetto i una variabile decisionale binaria $x_i \in \{0, 1\}$ (dove $x_i = 1$ significa “oggetto i scelto” e $x_i = 0$ “oggetto i scartato”), il problema si formula come:

$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq W, \\ & x_i \in \{0, 1\}, \quad i = 1, \dots, n. \end{aligned}$$

La funzione obiettivo somma i valori degli oggetti scelti; l'unico vincolo impone che il loro peso complessivo non ecceda la capacità. Senza perdita di generalità si assume $w_i \leq W$ per ogni i (un oggetto più pesante della capacità è scartabile a priori) e $\sum_i w_i > W$ (altrimenti la soluzione ottima banale seleziona tutti gli oggetti).

1.3 Perché lo Zaino 0/1

La scelta del problema è motivata da alcune proprietà che lo rendono adatto agli obiettivi dell'elaborato:

- è un problema-modello della PD: si mappa in modo pulito sulle quattro fasi del paradigma insegnate nel corso;
- ammette la variante di ottimizzazione spaziale $\Theta(nW) \rightarrow \Theta(W)$ richiesta esplicitamente dal Compito 2;
- ammette un algoritmo pseudo-polinomiale (il cui costo dipende dal valore di W , non solo da n): ciò rende lo studio empirico del Compito 3 ricco e non banale;
- si presta a un confronto con un solver ILP (Gurobi) come termine di paragone esterno per la validazione e i benchmark.

N.B. Il solver commerciale è stato utilizzato unicamente come benchmark di confronto e come oracolo di correttezza: il cuore dell'elaborato è l'algoritmo di programmazione dinamica.

2 Applicabilità della programmazione dinamica

La progettazione di un algoritmo di programmazione dinamica si articola in quattro fasi, di cui l'ultima è opzionale:

1. caratterizzazione della struttura di una soluzione ottima;
2. definizione ricorsiva del valore di una soluzione ottima;
3. calcolo del valore di una soluzione ottima con una strategia bottom-up;
4. costruzione di una soluzione ottima a partire dalle informazioni calcolate nella fase precedente.

Questa sezione svolge le prime due fasi per lo Zaino 0/1 e verifica intanto i due segni distintivi che sono anche i criteri di applicabilità del paradigma: l'esistenza di sottostrutture ottime all'interno di una soluzione ottima e la risoluzione ripetuta dello stesso sottoproblema.

2.1 I sottoproblemi

Applicare il paradigma richiede anzitutto di individuare una famiglia di sottoproblemi dell'istanza data. Nello Zaino 0/1 un sottoproblema è ancora un problema dello Zaino 0/1, ottenuto riducendo due grandezze: gli oggetti fra cui è ammesso scegliere e la capacità dello zaino. Fissato una volta per tutte un ordine sugli oggetti, si considerano i soli prefissi $\{1, \dots, i\}$: poiché su ciascun oggetto si decide una volta sola, esaminare gli oggetti in un ordine prestabilito non esclude alcuna soluzione.

Convenzione notazionale: per $0 \leq i \leq n$ e $0 \leq c \leq W$ si indica con $K[i, c]$ il valore di una soluzione ottima del sottoproblema costituito dai primi i oggetti con capacità c .

Il problema di partenza è il sottoproblema (n, W) e il valore cercato è dunque $K[n, W]$; i sottoproblemi distinti sono $(n+1)(W+1) = \Theta(nW)$.

2.2 Caratterizzazione della struttura di una soluzione ottima

Sia $S \subseteq \{1, \dots, i\}$ una soluzione ottima del sottoproblema (i, c) , con $i \geq 1$. Sull'oggetto i -esimo la domanda è una sola — lo prendo o non lo prendo? — e i due casi si trattano separatamente.

Caso $i \notin S$: l'oggetto i viene escluso. Allora $S \subseteq \{1, \dots, i-1\}$ ed è ammissibile per il sottoproblema $(i-1, c)$. È anche ottima per esso: se esistesse $S' \subseteq \{1, \dots, i-1\}$ ammissibile per $(i-1, c)$ e di valore maggiore, S' sarebbe ammissibile anche per (i, c) — stessa capacità, insieme di oggetti più ampio — contro l'ottimalità di S . Quindi $K[i, c] = K[i-1, c]$.

Caso $i \in S$: l'oggetto i viene incluso, possibile solo se $w_i \leq c$. Allora $S \setminus \{i\} \subseteq \{1, \dots, i-1\}$ ha peso complessivo non superiore a $c - w_i$ ed è quindi ammissibile per il sottoproblema $(i-1, c - w_i)$. È anche ottima per esso: se esistesse S' ammissibile per $(i-1, c - w_i)$ e di valore maggiore di quello di $S \setminus \{i\}$, allora $S' \cup \{i\}$ avrebbe peso non superiore a c , sarebbe ammissibile per (i, c) e avrebbe valore maggiore di quello di S , contro l'ottimalità di S . Quindi $K[i, c] = v_i + K[i-1, c - w_i]$.

In entrambi i casi l'argomento è del tipo taglia-e-incolla: si sostituisce una porzione della soluzione con una migliore e si ottiene una soluzione migliore di quella supposta ottima, il che è assurdo. Vale dunque l'esistenza di sottostrutture ottime all'interno di una soluzione ottima, primo segno distintivo e criterio di applicabilità del paradigma.

2.3 Definizione ricorsiva del valore di una soluzione ottima

I due casi appena discussi esauriscono le alternative. Quando $w_i > c$ l'oggetto i non entra nella capacità disponibile e resta il solo primo caso; quando $w_i \leq c$ valgono entrambi e si sceglie il valore maggiore. Con il caso base $K[0, c] = 0$ per ogni c — senza oggetti a disposizione il valore è nullo — si ottiene la ricorrenza

$$K[i, c] = \begin{cases} 0 & \text{se } i = 0, \\ K[i - 1, c] & \text{se } i \geq 1 \text{ e } w_i > c, \\ \max\{K[i - 1, c], v_i + K[i - 1, c - w_i]\} & \text{se } i \geq 1 \text{ e } w_i \leq c. \end{cases}$$

Il caso $c = 0$ non richiede una riga a sé: con capacità nulla nessun oggetto entra, essendo $w_i > 0$ per ipotesi, e la seconda riga riconduce $K[i, 0]$ a $K[0, 0] = 0$. Si osservi inoltre che $K[i, c] \geq K[i - 1, c]$ per ogni $i \geq 1$ e c , perché il massimo della terza riga contiene $K[i - 1, c]$: aggiungere un oggetto alla scelta non può peggiorare l'ottimo. La disuguaglianza tornerà utile nella ricostruzione della soluzione.

La ricorrenza determina il valore ottimo, che è unico, non l'insieme di oggetti che lo realizza: alla costruzione di una soluzione provvede la fase 4.

2.4 Risoluzione ripetuta dello stesso sottoproblema

Un algoritmo ricorsivo che traducesse alla lettera la ricorrenza può risolvere più volte lo stesso sottoproblema. Nel caso peggiore ogni chiamata si biforca e l'albero di ricorsione ha 2^n foglie, mentre i sottoproblemi distinti sono $(n + 1)(W + 1)$: appena 2^n supera tale quantità, qualche sottoproblema viene necessariamente affrontato più di una volta.

La coincidenza, del resto, non è un caso limite. Due chiamate ricorsive raggiungono lo stesso sottoproblema ogni volta che due sottoinsiemi distinti degli oggetti già decisi hanno lo stesso peso complessivo: già al livello $i - 2$, se $w_i = w_{i-1}$ e la capacità basta a entrambi, il sottoproblema $(i - 2, c - w_i)$ è raggiunto sia includendo l'oggetto i ed escludendo l' $(i - 1)$ -esimo, sia facendo il contrario. Con pesi interi limitati da W , tali collisioni sono la norma e non l'eccezione: è il secondo segno distintivo e criterio di applicabilità, noto in letteratura come sovrapposizione dei sottoproblemi.

La programmazione dinamica risponde a tale ridondanza risolvendo ciascun sottoproblema una volta sola e memorizzandone il valore in una tabella — è in questo senso un metodo di soluzione tabulare, applicabile a sottoproblemi non indipendenti, a differenza del divide-et-impera. Il costo cessa così di dipendere dal numero di cammini dell'albero di ricorsione e diventa proporzionale al numero di sottoproblemi distinti, come si vedrà nella Sezione 4.

3 I due algoritmi

Le fasi 3 e 4 del paradigma danno luogo ai due algoritmi richiesti dal Compito 1: il primo calcola il valore di una soluzione ottima riempiendo una tabella con strategia bottom-up, il secondo ricostruisce una soluzione ottima a partire dalla tabella così ottenuta.

3.1 Calcolo del valore di una soluzione ottima

La tabella K ha $n + 1$ righe e $W + 1$ colonne, indicizzate a partire da zero. La riga 0 è il caso base, le righe successive traducono la ricorrenza.

Knapsack-Value(w, v, n, W)

```

1  for  $c \leftarrow 0$  to  $W$ 
2      do  $K[0, c] \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $n$ 
4      do for  $c \leftarrow 0$  to  $W$ 
5          do if  $w[i] > c$   $\triangleright$  l'oggetto  $i$  ci sta in  $c$ ?
6              then  $K[i, c] \leftarrow K[i - 1, c]$   $\triangleright$  non ci sta: resta escluso
7              else  $K[i, c] \leftarrow \max\{K[i - 1, c], v[i] + K[i - 1, c - w[i]]\}$   $\triangleright$  meglio tra i due
8  return  $K$ 
```

Il valore ottimo dell'istanza è $K[n, W]$. L'ordine crescente delle righe garantisce che i valori letti alle righe 6 e 7 siano già stati calcolati; l'ordine delle colonne è invece indifferente, perché nessuna cella della riga i viene letta. Questa libertà cadrà nella variante a spazio ridotto. La tabella viene riempita per intero, comprese le celle che la ricorrenza non raggiungerebbe partendo da (n, W) .

3.2 Costruzione di una soluzione ottima

La tabella K contiene già l'informazione necessaria a risalire alle scelte compiute, e non occorre memorizzarle in una seconda tabella. Si procede a ritroso da (n, W) , una riga alla volta, tenendo aggiornata la capacità residua c .

Knapsack-Solution(K, w, n, W)

```

1   $S \leftarrow \emptyset$ 
2   $c \leftarrow W$ 
3  for  $i \leftarrow n$  downto 1
4      do if  $K[i, c] \neq K[i - 1, c]$ 
5          then  $S \leftarrow S \cup \{i\}$ 
6               $c \leftarrow c - w[i]$ 
7  return  $S$ 
```

Il test della riga 4 decide se l'oggetto i è stato incluso. Poiché $K[i, c] \geq K[i - 1, c]$, che i due valori differiscano significa $K[i, c] > K[i - 1, c]$; allora l'oggetto i appartiene a ogni soluzione ottima del sottoproblema (i, c) , altrimenti ne esisterebbe una contenuta in $\{1, \dots, i - 1\}$, ammissibile per $(i - 1, c)$ e di valore $K[i, c] > K[i - 1, c]$, contro la definizione di $K[i - 1, c]$. Se invece i due valori coincidono, esiste una soluzione ottima di (i, c) che non contiene l'oggetto i , ed è quella che l'algoritmo segue. Poiché le soluzioni ottime possono essere più d'una, l'algoritmo ne restituisce una.

4 Analisi asintotica

L'analisi che segue riguarda KNAPSACK-VALUE, il primo dei due algoritmi.

4.1 Tempo di esecuzione

Le righe 1–2 azzerano il caso base: $W + 1$ iterazioni di costo costante, dunque $\Theta(W)$. Le righe 3–7 sono due cicli annidati, n iterazioni esterne e $W + 1$ interne; il corpo esegue in entrambi i rami un numero costante di letture, confronti e assegnamenti, per un totale di $\Theta(nW)$. La riga 8 restituisce la tabella in tempo costante. Sommando,

$$T(n, W) = \Theta(W) + \Theta(nW) + \Theta(1) = \Theta(nW).$$

Si tratta di un Θ e non di un semplice O : il corpo del ciclo più annidato è eseguito esattamente $n(W + 1)$ volte qualunque siano pesi e valori, dunque il tempo è insieme $O(nW)$ e $\Omega(nW)$. Non c'è differenza fra caso migliore e caso peggiore; l'unico effetto dei dati è la scelta del ramo alle righe 6 o 7, che costano entrambe $\Theta(1)$.

4.2 Occupazione di memoria

La tabella ha $(n + 1)(W + 1)$ celle, dunque $\Theta(nW)$. Oltre a essa l'algoritmo usa soltanto qualche variabile scalare, $\Theta(1)$, mentre i vettori dei pesi e dei valori in ingresso occupano $\Theta(n)$: entrambi i contributi sono dominati dalla tabella. Lo spazio complessivo è quindi

$$S(n, W) = \Theta(nW).$$

Poiché nell'implementazione ogni cella è un intero a 64 bit (Sezione 5), l'occupazione della tabella è $(n + 1)(W + 1) \cdot 8$ byte a meno delle intestazioni degli array: una quantità nota a priori, che la Sezione 6 confronta con la memoria effettivamente misurata.

4.3 Natura pseudo-polinomiale

$\Theta(nW)$ è lineare nel *valore* della capacità, non nella lunghezza della sua codifica. Se W si scrive con b bit, allora W può valere fino a $2^b - 1$, e il tempo di esecuzione, pur essendo lineare in n , cresce come 2^b : non è limitato da alcun polinomio nella lunghezza dell'ingresso. L'algoritmo non è dunque polinomiale in senso stretto, ed è detto pseudo-polinomiale.

KNAPSACK-SOLUTION esegue n iterazioni di costo costante e occupa $O(n)$ per l'insieme restituito, ma richiede la tabella prodotta dal primo algoritmo: è questo a determinare il costo complessivo.

5 Implementazione e varianti

5.1 Struttura del codice

I due algoritmi della Sezione 3 e la variante a spazio ridotto sono codificati in tre classi che ne seguono lo pseudocodice riga per riga, con le sole differenze annotate più avanti; le restanti generano le istanze, controllano i risultati e prendono le misure.

- `KnapsackValue` — calcolo bottom-up della tabella;
- `KnapsackSolution` — ricostruzione di una soluzione ottima;
- `KnapsackRolling` — variante a spazio ridotto;
- `Instance` — generazione e lettura delle istanze da file;
- `BruteForce` — ricerca esaustiva sui sottoinsiemi, riscontro sulle istanze piccole;

- **CorrectnessTest** — batteria di prove di correttezza;
- **Bench** — esecuzione ripetuta e raccolta delle misure;
- **Main** — programma a riga di comando.

L’elaborazione è interamente batch, con ingresso e uscita da file.

La correttezza è verificata da una batteria di 305 test che, su istanze generate con parametri e semi diversi, confronta fra loro il valore ottimo dei tre algoritmi e controlla che la soluzione ricostruita sia ammissibile e valga l’ottimo.

5.2 La variante a spazio ridotto

Il Compito 2 chiede varianti che migliorino le prestazioni spaziali. Quella adottata qui è nota come *rolling array*, ed è con questo nome che compare nelle figure. Nel calcolo di $K[i, c]$ intervengono soltanto celle della riga $i - 1$: le righe precedenti, una volta usate, non servono più. Se ne può quindi tenere una sola, e aggiornarla in luogo.

Knapsack-Rolling(w, v, n, W)

```

1  for  $c \leftarrow 0$  to  $W$ 
2      do  $K[c] \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $n$ 
4      do for  $c \leftarrow W$  downto  $w[i]$ 
5          do if  $K[c - w[i]] + v[i] > K[c]$ 
6              then  $K[c] \leftarrow K[c - w[i]] + v[i]$ 
7  return  $K[W]$ 
```

Il verso decrescente della riga 4 garantisce che $K[c - w[i]]$ contenga ancora il valore dell’iterazione precedente, cioè $K[i - 1, c - w_i]$. Percorrendo c in senso crescente si leggerebbe un valore già aggiornato nell’iterazione corrente, e l’oggetto i finirebbe per essere incluso più volte.

Lo spazio scende da $\Theta(nW)$ a $\Theta(n + W)$: $\Theta(W)$ di memoria di lavoro oltre ai vettori d’ingresso, che per $W \gg n$ si riduce di fatto a $\Theta(W)$. Il tempo resta $O(nW)$, ma non più $\Theta(nW)$: il ciclo interno si ferma a $w[i]$, quindi il numero di iterazioni dipende dai pesi. Con pesi uniformi in $[1, W]$ ne esegue in media metà, ed è questa la ragione principale per cui nelle misure la variante risulta da due a tre volte più veloce della tabella pur avendo lo stesso ordine di grandezza asintotico.

Il prezzo è la perdita della ricostruzione: senza le righe precedenti il confronto $K[i, c] \neq K[i - 1, c]$ non è più eseguibile, e della soluzione resta il solo valore.

5.3 Scelte tecnologiche e implementative

Il linguaggio scelto è Java, versione 21: è abbastanza veloce da reggere istanze grandi ed è disponibile un’API ufficiale di Gurobi. Il modulo che chiama il solver è separato e si compila solo se la libreria è presente, così che il resto resti eseguibile senza.

Le celle della tabella sono interi a 64 bit. Il formato d’ingresso non pone limiti ai valori, e la loro somma può superare la capienza di un intero a 32 bit: in Java l’eccedenza non produce alcun errore, e si otterrebbe un risultato sbagliato senza avvisi. La scelta raddoppia l’occupazione rispetto a un intero a 32 bit, e i conti della Sezione 4 ne tengono conto.

Il caso base non richiede codice: in Java gli array sono azzerati alla creazione, e la riga 0 contiene quindi già 0 in ogni cella.

6 Sperimentazione

6.1 Istanze, metodo e piattaforma

Le istanze sono generate automaticamente: i pesi sono interi estratti a caso in modo uniforme fra 1 e W , i valori interi fra 1 e 1000; il seme del generatore dipende dal parametro che varia, così che ogni punto delle curve sia riproducibile. Per separare i due fattori di crescita si eseguono due campagne distinte: nella prima n cresce da 1000 a 10 000 a passo 1000 con W fermo a 1000, nella seconda è W a crescere nello stesso intervallo con n fermo a 1000. Poiché i pesi sono estratti in $[1, W]$, nella seconda campagna crescono insieme alla capacità: le istanze restano di difficoltà costante, e di questo si tiene conto nel commento dei risultati.

Ogni punto è il minimo di sette esecuzioni cronometrate, precedute da tre di riscaldamento, perché il codice appena avviato è ancora interpretato e non compilato; per il solver, che ha un costo di avvio proprio, si prendono tre soluzioni dopo una di riscontro e si legge il solo tempo di soluzione, escludendo la costruzione del modello. Si prende il minimo e non la media perché è la misura meno esposta alle pause del gestore della memoria. L'occupazione di memoria è registrata due volte: calcolata dalla formula e misurata sull'heap, come mediana di tre letture prese dopo una raccolta dei rifiuti.

Le misure provengono da un portatile con processore Apple M1 e sistema macOS 26.5.2, macchina virtuale Java 21. Gli sweep della programmazione dinamica girano con heap fissato a 2 GB, valore minimo uguale al massimo per evitare ridimensionamenti durante le misure. Il solver di confronto è Gurobi 12.0.3, configurato con un solo thread, come gli algoritmi in esame, e con gap di ottimalità nullo, perché lo si usa come oracolo esatto e non come euristica.

6.2 Crescita empirica e analisi asintotica

Nella Figura 1 il tempo dei due algoritmi di programmazione dinamica cresce in proporzione a n , come previsto da $\Theta(nW)$ a capacità fissa, mentre quello di Gurobi cresce in modo irregolare e non monotono, con oscillazioni ampie.

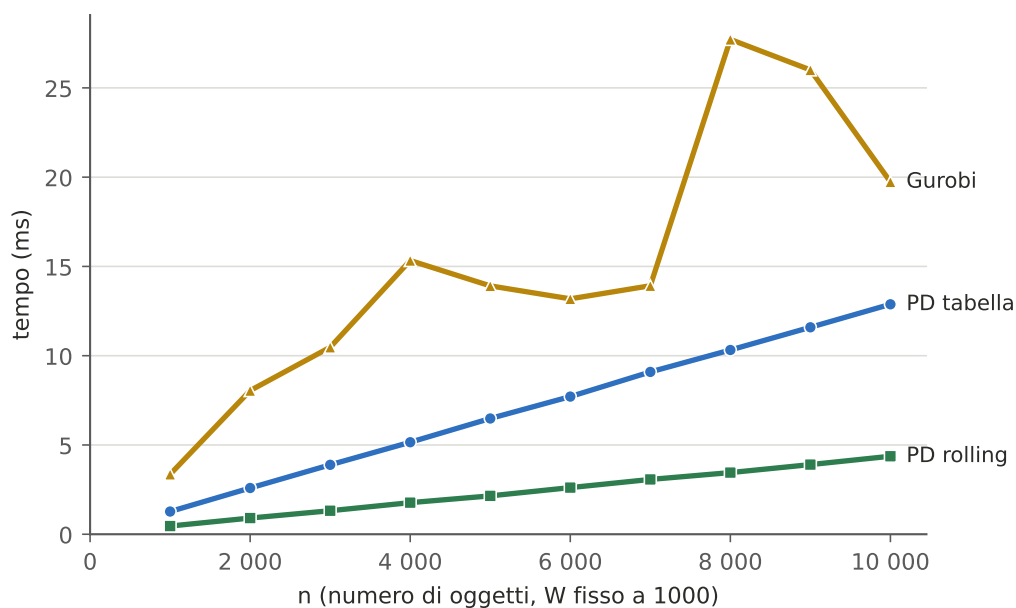


Figura 1: Tempo di esecuzione al crescere del numero di oggetti, a capacità fissa $W = 1000$.

Nella Figura 2 la programmazione dinamica ci mette sempre più tempo al crescere della capacità, mentre Gurobi resta pressoché costante. La ragione è strutturale: Gurobi cerca con il branch and bound sul rilassamento continuo, dove W entra una sola volta, come coefficiente nel vincolo di peso, e crescerlo non comporta più lavoro di ricerca. La programmazione dinamica invece deve costruire una tabella larga $W + 1$ colonne: crescere W significa fisicamente più celle da calcolare, un costo meccanico legato alla dimensione della tabella — è la manifestazione empirica della natura pseudo-polinomiale dell'algoritmo, il cui tempo dipende dal valore di W e non dalla sua lunghezza.

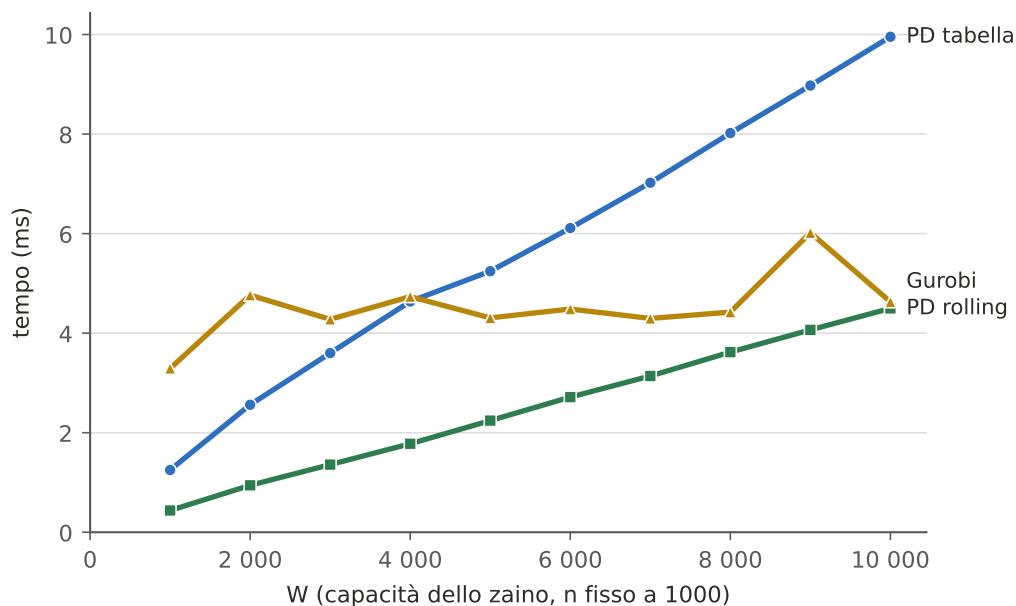


Figura 2: Tempo di esecuzione al crescere della capacità, a numero di oggetti fisso $n = 1000$.

La linearità non è soltanto un'impressione visiva: interpolando le misure con una retta ai minimi quadrati, R^2 vale 0,9999 e 0,9966 per l'algoritmo base (su n e su W) e 0,9997 e 0,9998 per la variante a spazio ridotto. Più probante è il tempo per cella — la pendenza della retta divisa per il fattore tenuto fisso: 0,43 e 0,45 ns per la variante a spazio ridotto, lo stesso valore in due campagne indipendenti; per l'algoritmo base i due valori si discostano, 1,3 contro 0,9, verosimilmente per la struttura a righe separate della tabella in Java — a parità di celle, dieci volte più righe costano di più.

Quanto alla memoria, nella Figura 3 i valori misurati sull'heap seguono la formula $(n+1)(W+1) \cdot 8$ byte, con uno scarto di pochi punti percentuali dovuto alla granularità di mezzo megabyte con cui la macchina virtuale alloca: le letture sono multipli esatti di quel valore, e lo scarto tende a ridursi al crescere della tabella. La previsione dell'analisi asintotica è quindi confermata anche nella costante, non solo nell'andamento.

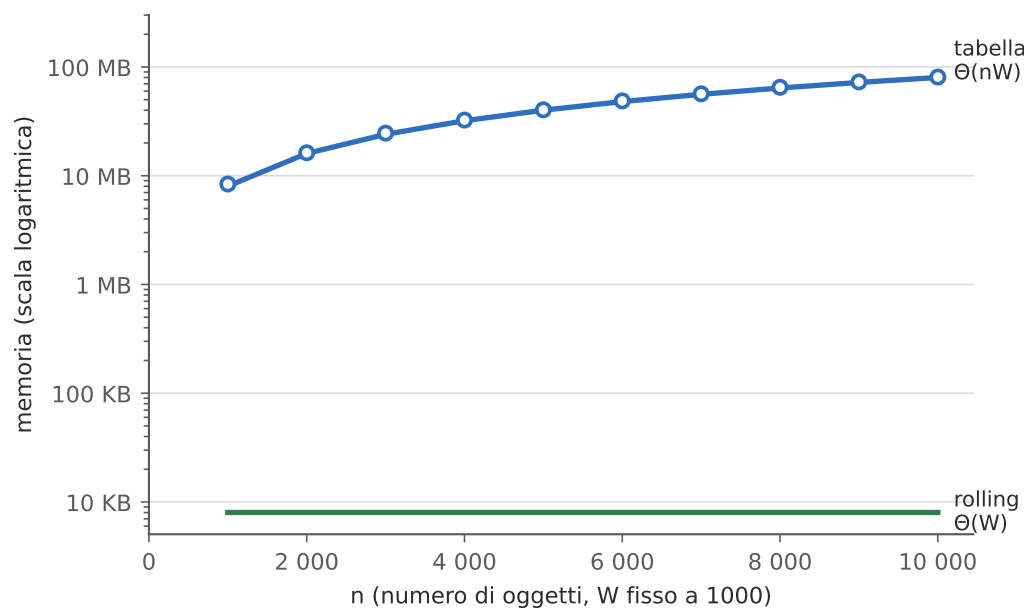


Figura 3: Occupazione di memoria della tabella, calcolata dalla formula (linea) e misurata sull'heap (cerchi), a confronto con quella della variante a spazio ridotto, di cui si riporta la sola formula (Sezione 7.1).

6.3 Limiti pratici

La variante a spazio ridotto non serve a essere più veloce, ma a rendere trattabili istanze che altrimenti non lo sarebbero. Fissati $n = 2000$ e un tetto di memoria, si è fatta crescere la sola capacità; a differenza delle curve, ciascuna riga della Tabella 1 è una singola esecuzione a freddo, su una macchina virtuale appena avviata.

Tabella 1: Memoria richiesta e tempo al crescere di W , con $n = 2000$. La tabella completa dispone di 2 GB di heap, la variante a spazio ridotto di 1 GB. I prefissi sono binari.

W	tabella $\Theta(nW)$		a spazio ridotto	
	memoria	tempo	memoria	tempo
100 000	1,5 GB	0,81 s	781 KB	0,09 s
300 000	4,5 GB	memoria esaurita	2,3 MB	0,28 s
1 000 000	14,9 GB	memoria esaurita	7,6 MB	0,93 s

Già a $W = 300\,000$ l'algoritmo base non arriva in fondo, mentre la variante risolve la stessa istanza usando 2,3 MB e 0,28 s. È il risultato richiesto dal Compito 2: a parità di tempo asintotico, la riduzione dello spazio estende di almeno un ordine di grandezza, sui casi provati, le capacità trattabili.

Il collo di bottiglia è dunque la memoria, non il tempo. Lo si vede nella stessa riga della tabella: a $W = 300\,000$ l'algoritmo base esaurisce l'heap, ma la variante a spazio ridotto — stessa complessità temporale, stesso calcolo — risolve l'istanza in appena 0,28 s. Il tempo non era mai vicino a diventare un problema: è la memoria ad arrivare prima al limite.

6.4 Confronto con il solver

Su tutti i venti punti delle due campagne la tabella completa e la variante a spazio ridotto danno lo stesso valore ottimo, e questo coincide con quello del solver; il valore è registrato nei file CSV accanto ai tempi. Il riscontro esterno di correttezza è dunque soddisfatto. Sui tempi nessuno dei due domina l'altro. La programmazione dinamica ha un tempo prevedibile, determinato da n e W , mentre quello del solver dipende da quanto l'istanza si presta ai tagli e alle riduzioni del branch and bound. Al crescere della capacità le parti si invertono (Figura 2): su istanze non correlate come queste il tempo del solver resta pressoché ininfluente, mentre quello della programmazione dinamica cresce comunque, per il motivo già visto.

6.5 Riproduzione degli esperimenti

Il codice si compila con lo script fornito e non richiede altro che una macchina virtuale Java. Gurobi è opzionale: `compile.sh` include il modulo che lo interroga solo se trova `gurobi.jar` (variabile `GUROBI_HOME` o `lib/gurobi.jar`), e `campagna.py` salta automaticamente le curve del solver quando non lo trova, riproducendo comunque per intero la parte di programmazione dinamica. I comandi principali sono i seguenti.

```
./compile.sh                compila in bin/
java -cp bin knapsack.Main test  batteria di correttezza

java -cp bin knapsack.Main generate 200 5000 42 -o istanza.txt
java -cp bin knapsack.Main solve istanza.txt --dump-table tabella.csv
java -cp bin knapsack.Main solve istanza.txt --rolling

java -Xms2g -Xmx2g -cp bin knapsack.Main bench nsweep \
    --fixed 1000 --from 1000 --to 10000 --step 1000 --reps 7 -o dp_n.csv

./campagna.py                rifa l'intera campagna in data/campagna/
./grafici.py                 rifa le figure di questa relazione
```

L'opzione `--dump-table` salva la tabella K in formato CSV, così che sia disponibile per l'ispezione richiesta dalla traccia. I file CSV della campagna fanno parte del materiale consegnato.

7 Limitazioni e conclusioni

7.1 Limitazioni riscontrate

La misura della memoria sull'heap ha una risoluzione dell'ordine del mezzo megabyte, imposta dalla granularità con cui la macchina virtuale alloca: sufficiente per la tabella, che occupa decine di megabyte, inutilizzabile per la variante a spazio ridotto, i cui pochi kilobyte restano sotto la soglia. Di quest'ultima si riporta perciò la sola occupazione calcolata.

La variante a spazio ridotto restituisce il solo valore ottimo: per ottenere anche l'insieme degli oggetti scelti occorre la tabella completa, e quindi la memoria che la variante serve a risparmiare.

I pesi sono estratti in $[1, W]$: al crescere della capacità crescono con essa, e nella seconda campagna la soluzione ottima resta composta da pochi oggetti. Le istanze non diventano dunque mai più difficili, e la piattezza della curva del solver va letta anche alla luce di questo. Non sono state generate istanze costruite apposta per mettere in difficoltà il solver: il confronto

va quindi letto per quello che è, un termine di paragone esterno su istanze casuali, e non come uno studio sistematico del solver.

Le misure provengono infine da una sola macchina e da una sola macchina virtuale: gli andamenti sono affidabili, i valori assoluti no.

7.2 Conclusioni

I tre compiti sono stati svolti sul problema dello Zaino 0/1: la programmazione dinamica è applicabile perché valgono entrambi i criteri; i due algoritmi sono stati scritti, analizzati e codificati; la sperimentazione ne ha confrontato la crescita empirica con l'analisi asintotica.

Due sono i risultati principali. Da un lato l'analisi prevede il comportamento osservato con precisione: il tempo cresce linearmente in ciascuno dei due fattori e la memoria segue la formula. Dall'altro il fattore che decide quali istanze siano davvero trattabili non è il numero di oggetti ma la capacità, ed è sulla memoria, non sul tempo, che il limite si presenta per primo: da qui l'utilità concreta della variante a spazio ridotto.